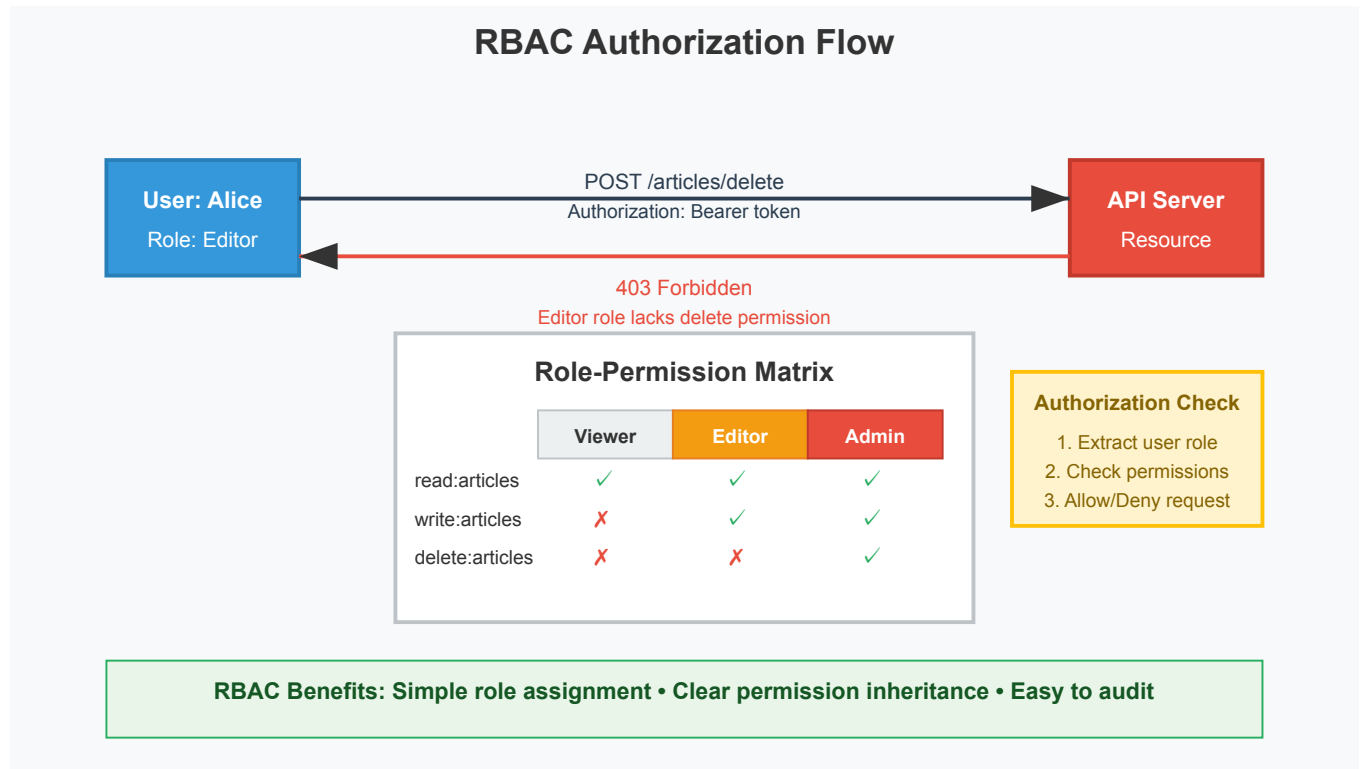


API Authorization

 systemdesignschool.io/fundamentals/api-design-authorization



Authorization determines what authenticated users can access. After verifying identity (authentication), systems check permissions to decide which resources and actions are allowed for each user.

Authorization vs Authentication

Authentication answers "Who are you?" Authorization answers "What can you do?" These are distinct security layers that work together. A user might successfully log in (authentication) but lack permission to delete files (authorization).

How they work together

Google Drive demonstrates this clearly: logging in with your Google account proves your identity, but accessing a shared document depends on whether you have viewer, commenter, or editor permissions. The API checks both your identity and your specific permissions for each resource.

RBAC: Role-Based Permissions

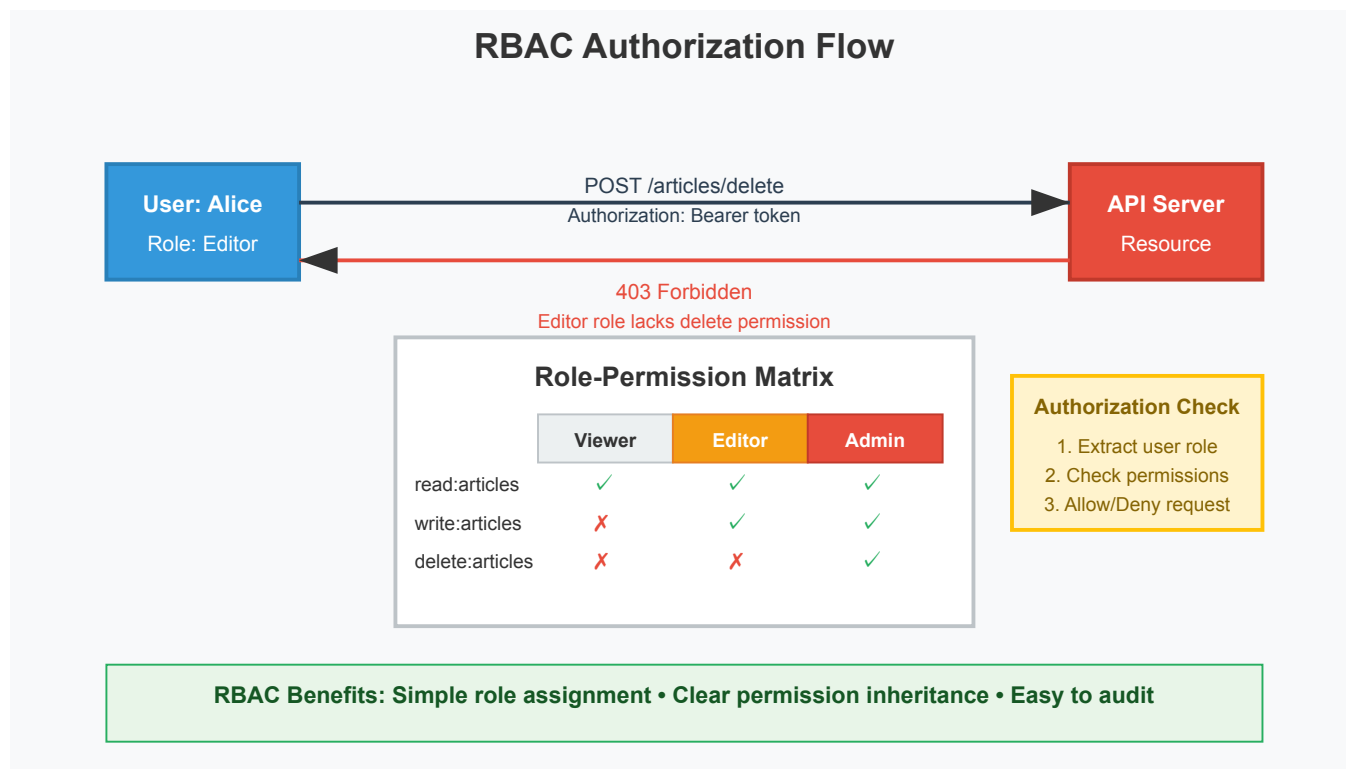
Role-based access control assigns users to roles like admin, editor, or viewer. Each role bundles specific permissions, making access management scalable for large organizations.

Simple but effective

Most business applications use RBAC because it maps naturally to organizational structures. GitHub uses repository permission levels (read, triage, write, maintain, admin) where each level includes specific permissions like pushing code, managing issues, or configuring settings. Users inherit all permissions from their assigned level.

Production patterns

Slack organizes workspace permissions around roles: the Primary Owner can delete the workspace, admins manage users and settings, members can create channels, and guests have limited access to assigned channels. This hierarchy prevents privilege escalation while keeping management simple.



RBAC Implementation with Role Hierarchy

```
from typing import Set, Dict, List
from dataclasses import dataclass

READ_POSTS = "read:posts"
WRITE_POSTS = "write:posts"
DELETE_POSTS = "delete:posts"
MANAGE_USERS = "manage:users"
ADMIN_SETTINGS = "admin:settings"
MODERATOR = "moderator"

# Define role hierarchies and permissions
self.role_permissions: Dict[Role, Set[Permission]] = {
    Role.VIEWER: {Permission.READ_POSTS},
```

```
Role.EDITOR: {Permission.READ_POSTS, Permission.WRITE_POSTS},
Role.MODERATOR: {Permission.READ_POSTS, Permission.WRITE_POSTS,
Permission.DELETE_POSTS},
Role.ADMIN: {Permission.READ_POSTS, Permission.WRITE_POSTS,
Permission.DELETE_POSTS, Permission.MANAGE_USERS,
Permission.ADMIN_SETTINGS}
# Role hierarchy (higher roles inherit lower role permissions)
self.role_hierarchy: Dict[Role, Set[Role]] = {
Role.ADMIN: {Role.MODERATOR, Role.EDITOR, Role.VIEWER},
```

ABAC: Attribute-Based Control

Attribute-based access control makes decisions using multiple attributes: user properties, resource characteristics, environmental factors, and business rules. This enables fine-grained, context-aware authorization.

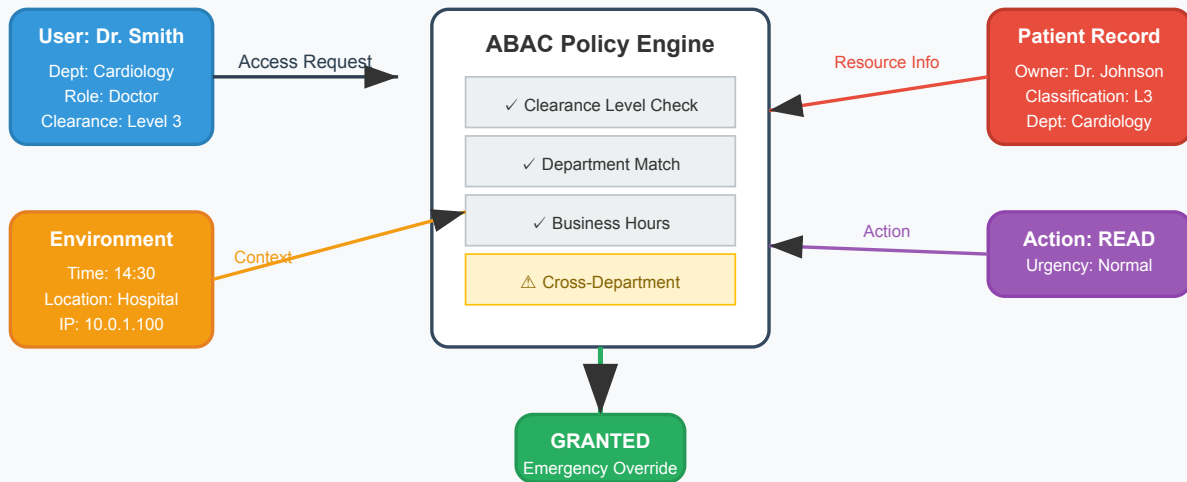
Dynamic policy decisions

Healthcare systems use ABAC extensively. A doctor can access patient records during their shift, in their assigned department, for patients under their care. The same doctor might be denied access to the same record outside work hours or from an untrusted location.

Complex but flexible

Financial services leverage ABAC for regulatory compliance. A trader can execute transactions up to their authorized limit, during market hours, for their assigned asset classes, while logged in from approved locations. These policies adapt dynamically based on current context.

ABAC Authorization Flow



Dynamic Policy Examples

Healthcare
Access during shift hours
Emergency overrides allowed

Banking
Location-based restrictions
Time & amount limits

Cloud Services
Project & resource scoping
Cost center validation



ABAC with Context-Aware Policies

```
from dataclasses import dataclass
from datetime import datetime, time
from typing import Dict, Any, Callable

security_clearance: int
manager_id: str = None

class ResourceAttributes:
    classification: str # public, internal, confidential, secret

class EnvironmentAttributes:
    time_of_access: datetime
    action: str # read, write, delete, share
    urgency: str = "normal" # normal, urgent, emergency

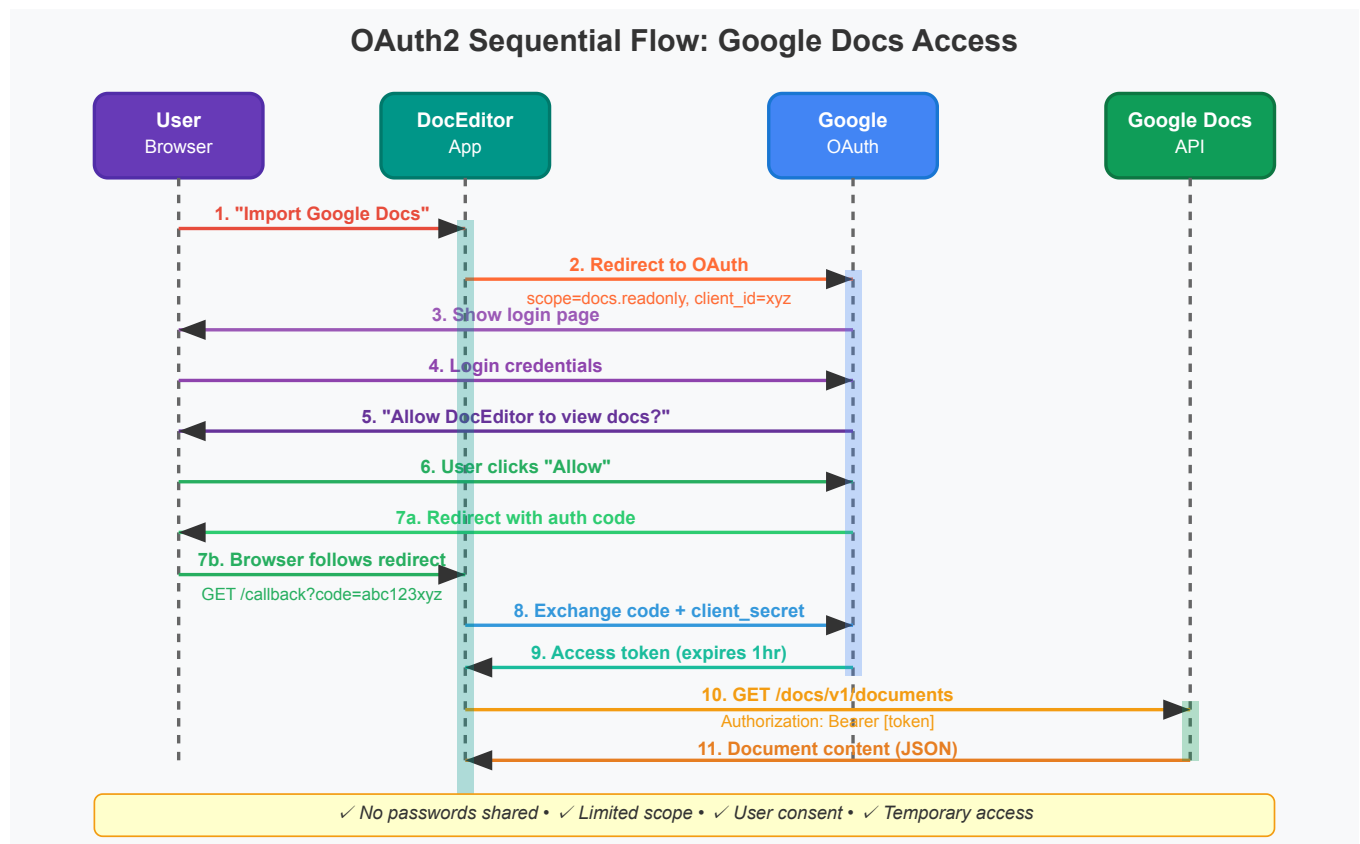
"""Attribute-based access control policy engine"""
```

OAuth: Delegated Authorization

OAuth enables applications to access user resources without sharing credentials. Users grant specific permissions (scopes) to third-party applications, which receive tokens limited to those approved actions.

Scope-based permissions

When you connect Spotify to Last.fm, you grant scopes like "read your currently playing track" and "read your playlists" but not "modify your playlists." Spotify's API enforces these scope limitations on every request, ensuring Last.fm can only perform authorized actions.



Production implementation

Twitter's API uses OAuth scopes extensively. A social media management tool might request "read tweets," "post tweets," and "read direct messages" but not "delete account." Users see exactly what permissions they're granting, and Twitter's API validates scopes on each request.

OAuth Scope Validation

```
from typing import Set, Dict, Optional
from dataclasses import dataclass
from datetime import datetime, timedelta

class OAuthScopeValidator:
    """OAuth scope validation and token management"""
    # Define available scopes and their hierarchies
    self.available_scopes = {
        "read:profile", "write:profile",
        "read:posts", "write:posts", "delete:posts",
        "read:messages", "write:messages",
        "admin:users", "admin:system"
```

```
# Scope hierarchies (higher scopes include lower ones)
self.scope_hierarchy = {
  "write:profile": {"read:profile"},
  "write:posts": {"read:posts"},
  "delete:posts": {"read:posts", "write:posts"},
  "write:messages": {"read:messages"},
  "admin:users": {"read:profile", "write:profile"},
  "admin:system": {"admin:users", "read:profile", "write:profile"}
self.active_tokens: Dict[str, OAuthToken] = {}
```

Choosing Your Authorization Strategy

Different authorization patterns suit different use cases. RBAC works for most business applications with clear organizational roles. ABAC handles complex compliance requirements and dynamic policies. OAuth enables secure third-party integrations.

Decision factors

Pattern	Complexity	Flexibility	Use Cases
RBAC	Low	Medium	Internal apps, clear hierarchies
ABAC	High	Very High	Compliance, dynamic policies
OAuth	Medium	Medium	Third-party access, API platforms

Hybrid approaches

Many systems combine authorization patterns. GitHub uses RBAC for repository permissions, OAuth scopes for API access, and attribute-based rules for security policies (like requiring 2FA for sensitive operations). This layered approach provides both simplicity and flexibility where needed.